

Program Language Constructs

Higher-order functions

Peter Sestoft

Niels Hallenberg

Monday 2026-07-24



Plan for today

- Higher-order functions in F#
- A higher-order functional language

Higher-order functions and anonymous functions in F#

- A higher-order function takes another function as argument

```
let rec map f xs =  
    match xs with  
    | []      -> []  
    | x::xr  -> f x :: map f xr
```

$('a \rightarrow 'b) \rightarrow ('a \text{ list} \rightarrow 'b \text{ list})$

```
let mul2 x = 2.0 * x;;  
map mul2 [4.0; 5.0; 89.0];;
```

[8.0; 10.0; 178.0]

- Anonymous functions

```
map (fun x -> 2.0 * x) [4.0; 5.0; 89.0]
```

[false; false; true]

```
map (fun x -> x > 10.0) [4.0; 5.0; 89.0]
```

Function types in C#

- Delegate types

```
delegate R Func<R>()  
delegate R Func<A1,R>(A1 x1)  
delegate R Func<A1,A2,R>(A1 x1, A2 x2)
```

unit -> R

A1 -> R

A1 * A2 -> R

```
delegate void Action<A1>(A1 x1)  
delegate void Action<A1,A2>(A1 x1, A2 x2)
```

A1 -> unit

A1*A2 -> unit

- Anonymous method expressions

```
delegate(int x) { return x>10; }
```

Func<int,bool>

```
delegate(int x) { return x*x; }
```

Func<int,int>

```
(int x) => x>10  
x => x>10  
x => x*x
```

C# 3.0

```
fun (x:int) -> x>10  
fun x -> x>10  
fun x -> x*x
```

F#

Uniform iteration over a list

```
let rec sum xs =  
  match xs with  
  | []      -> 0  
  | x::xr   -> x + sum xr
```

int list -> int

```
let rec prod xs =  
  match xs with  
  | []      -> 1  
  | x::xr   -> x * prod xr
```

- Generalizing 0/1 to e, and +/* to f:

```
let rec foldr f xs e =  
  match xs with  
  | []      -> e  
  | x::xr   -> f x (foldr f xr e)
```

('a -> 'b -> 'b) ->
'a list -> 'b -> 'b

List.foldBack in F#

The foldr function replaces :: by f, and [] by e:

$$\text{foldr } \diamond (x_1::x_2::\dots::x_n::[]) e = x_1 \diamond (x_2 \diamond (\dots \diamond (x_n \diamond e) \dots))$$

Higher-order micro-ML/micro-F#

- Higher-order functional language
 - A function may be given as argument:

```
let twice g x = g(g x)
```

- A function may be returned as result

```
let add x = let f y = x+y in f
let addtwo = add 2
let x = 77
addtwo 5
```

add has two arguments!

- Closures are needed:
 - The function returned must enclose the value of f's parameter x – has nothing to do with later x
- Same micro-ML syntax: Fun/Absyn.fs

Interpretation of a higher-order language

- The closure machinery is already in place (HigherFun.fs):

```
type value =  
  | Int of int  
  | Closure of string * string * expr * value env  
              (* (f,      x,      fBody, fDeclEnv) *)  
  
let rec eval (e : expr) (env : value env) : value =  
  match e with  
  | ...  
  | Letfun(f, x, fBody, letBody) ->  
    let bodyEnv = (f, Closure(f, x, fBody, env)) :: env  
    eval letBody bodyEnv
```

The function `f` can be called from `letBody`, and must be in the `bodyEnv`.

Interpretation of a higher-order language

- Just redefine function application:

```
let rec eval (e : expr) (env : value env) : value =  
  match e with  
  | ...  
  | Call(eFun, eArg) ->  
    let fClosure = eval eFun env  
    match fClosure with  
    | Closure (f, x, fBody, fDeclEnv) ->  
      let xVal = eval eArg env  
      let fBodyEnv =  
        (x, xVal) :: (f, fClosure) :: fDeclEnv  
      eval fBody fBodyEnv  
    | _ -> failwith "eval Call: not a function"
```

Was
lookup env eFun in Fun.fs.

The
function *f*
may call
itself
recursively